

# Operating Systems

No. 4

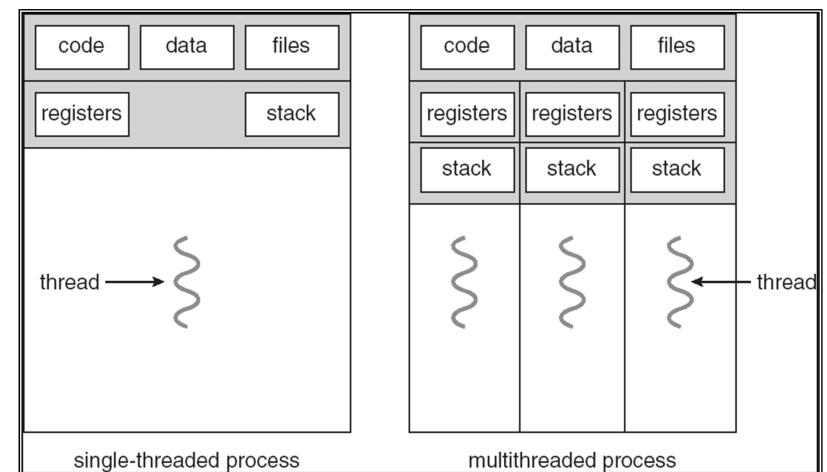
Sarun Intakosum

## Thread

### Threads Concepts

- **Thread:** A portion of a program that can run independently of other portions.
  - A thread is a sequence of instructions that runs independently within a process, with its own program counter, system registers, and stack.
  - Thread is also called **lightweight process**.
  - Thread becomes the element that uses the CPU and is scheduled for execution
  - Swapping threads is less time consuming than swapping processes
- Multithreaded applications programs can have several threads running at one time with the same or different priorities

### Single and Multithreaded Processes



# Categories of OS

- Single-process single-threaded
  - DOS
- Single-Process multi-threaded
  - JVM *↓ process can have many thread.*
- Multiprocessing single-threaded
  - Tradition UNIX
- Multiprocessing multi-threaded
  - Modern Operating Systems => Windows, Linux, MacOS

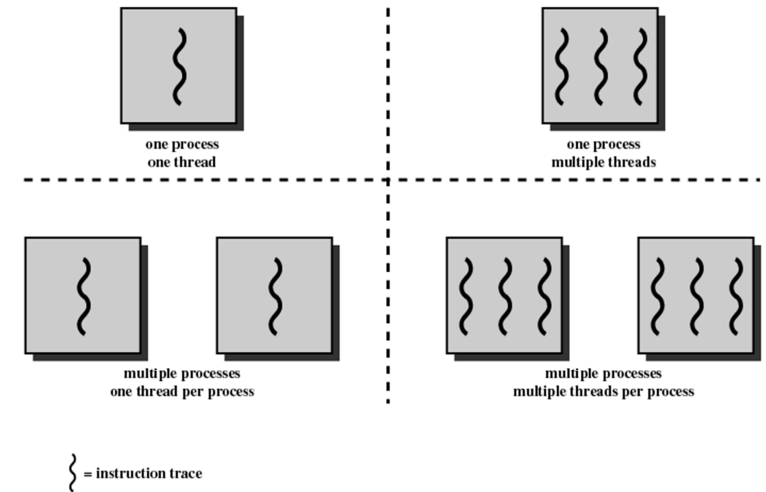
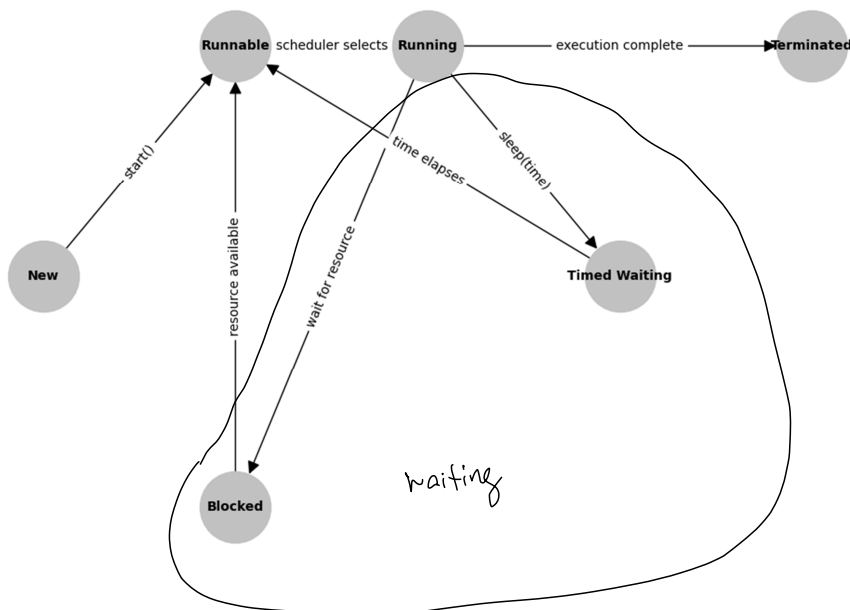


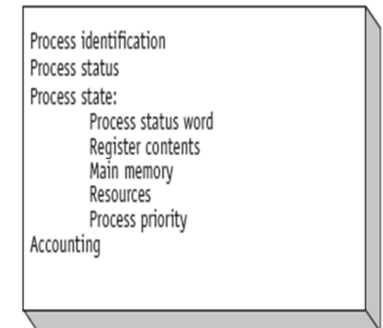
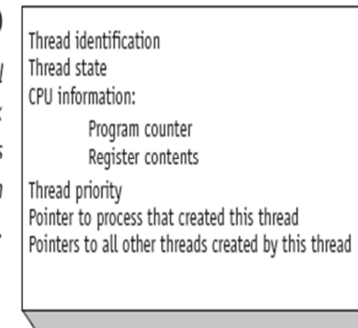
Figure 4.1 Threads and Processes [ANDE97]

# Thread States



# Thread Control Block

(figure 6.7)  
Comparison of a typical  
Thread Control Block  
(TCB) vs. a Process  
Control Block (PCB) from  
Chapter 4.



## Types of Threads

There are two general types of threads:

- User-level threads (ULT) *handle at User level*
- Kernel-level threads (KLT).

## User Threads

- Thread management done by user-level threads library
- Three primary thread libraries:
  - POSIX Pthreads
  - Win32 threads
  - Java threads
    - Generally implemented using a thread library available on host OS.
- This library enables the program to call the necessary functions for managing threads, including establishing, suspending, and terminating them.

*JVM [ windows - win32, linux - posix ]*

## Kernel Threads

- Supported by the Kernel
- Examples
  - Modern operating systems

*mapping between user level → kernel level*

## Multithreading Models

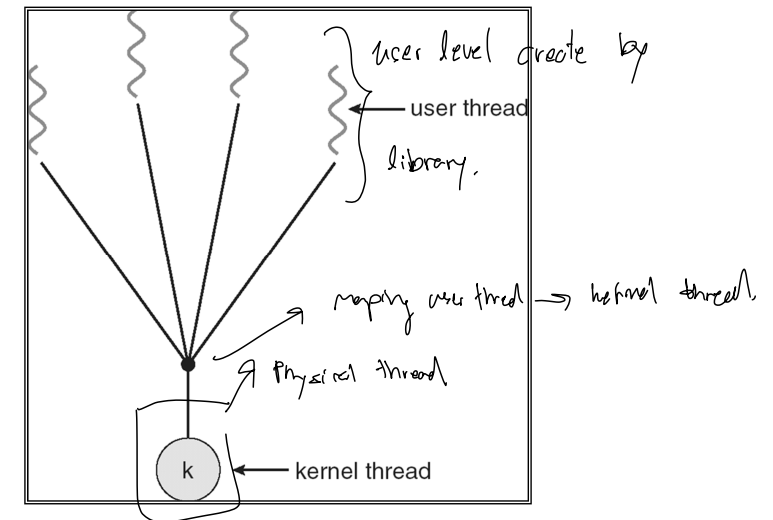
- Many-to-One *Many user → One thread*
- One-to-One
- Many-to-Many

## Many-to-One

- Many user-level threads mapped to single kernel thread
- Examples:
  - Early Unix System

user  $\rightarrow$  library  
 kernel  $\rightarrow$  OS

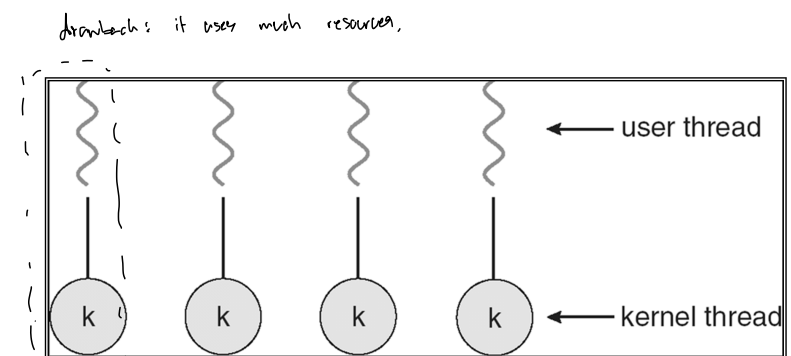
## Many-to-One Model



## One-to-One

- Each user-level thread maps to kernel thread
- Examples
  - Windows starting from NT
  - Linux
  - Solaris 9 and later

## One-to-one Model

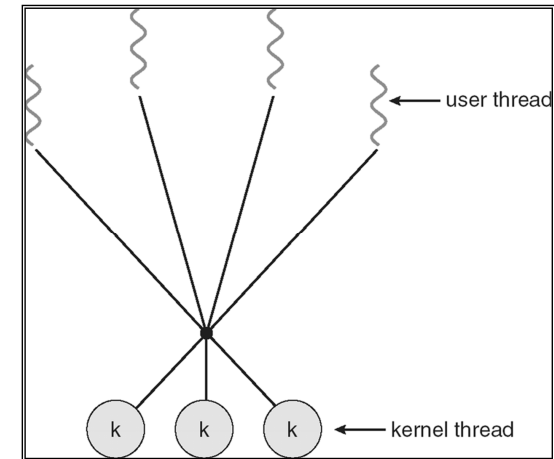


Suppose if the thread do nothing, it will just waiting the resource.

benefit: easy to manage.

## Many-to-Many Model

- Allows many user level threads to be mapped to many kernel threads
- Allows the operating system to create a sufficient number of kernel threads
- Examples
  - Solaris prior to version 9
  - Tru64 UNIX



## Pthreads

```
int sum; /* this data is shared by the thread(s) */
void *runner(void *param); /* the thread */
main(int argc, char *argv[])
{
    pthread_t tid; /* the thread identifier */
    pthread_attr_t attr; /* set of attributes for the thread */
    pthread_attr_init(&attr); /* get the default attributes */
    /* create the thread */
    pthread_create(&tid, &attr, runner, argv[1]);
    /* now wait for the thread to exit */
    pthread_join(tid, NULL); wait for the process id
    printf("sum = %d\n", sum);
}

void *runner(void *param) {
    int upper = atoi(param);
    int i; convert string to int
    sum = 0;
    if (upper > 0) {
        for (i = 1; i <= upper; i++)
            sum += i;
    }
    pthread_exit(0);
}
```

## Java Threads

- Java threads are managed by the JVM
- Java threads may be created by:
  - Extending Thread class
  - Implementing the Runnable interface

## Extending the Thread Class

```
class Worker1 extends Thread
{
    public void run() {
        System.out.println("I Am a Worker Thread");
    }
}

public class First
{
    public static void main(String args[]) {
        Worker1 runner = new Worker1();
        runner.start();

        System.out.println("I Am The Main Thread");
    }
}
```

## The Runnable Interface

```
public interface Runnable
{
    public abstract void run();
}
```

## Implementing the Runnable Interface

```
class Worker2 implements Runnable
{
    public void run() {
        System.out.println("I Am a Worker Thread ");
    }
}

public class Second
{
    public static void main(String args[]) {
        Runnable runner = new Worker2();
        Thread thrd = new Thread(runner);
        thrd.start();

        System.out.println("I Am The Main Thread");
    }
}
```

## Joining Threads

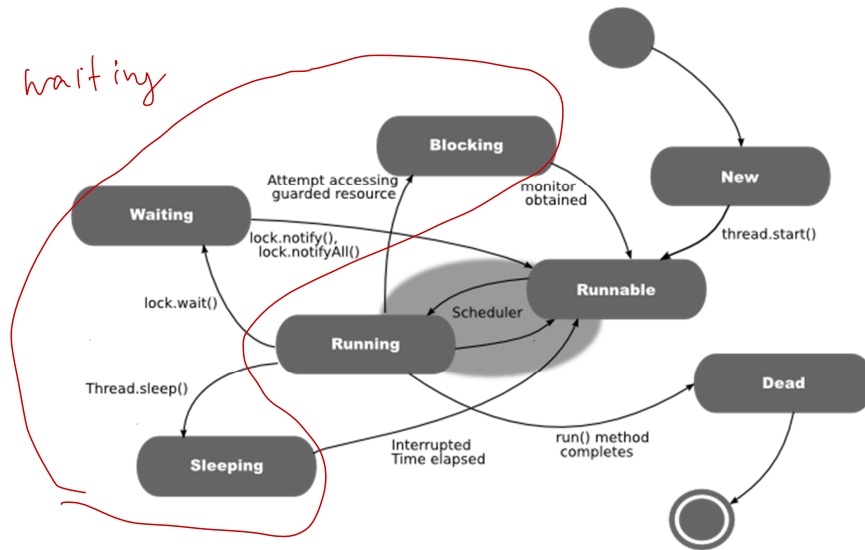
```
class JoinableWorker implements Runnable
{
    public void run() {
        System.out.println("Worker working");
    }
}

public class JoinExample
{
    public static void main(String[] args) {
        Thread task = new Thread(new JoinableWorker());
        task.start();

        try { task.join(); }
        catch (InterruptedException ie) { }

        System.out.println("Worker done");
    }
}
```

## Java Thread States



<https://www.pngegg.com/en/png-tvayp>

## Thread Cancellation

- Terminating a thread before it has finished
- Two general approaches:
  - Asynchronous cancellation** terminates the target thread immediately
  - Deferred cancellation** allows the target thread to periodically check if it should be cancelled

*pthread* {  
 ↓  
 default is deferred, consider suitable.

*pthread\_t* - set cancel state (PTHREAD\_CANCEL\_DISABLE)

## Thread Cancellation

```
Thread thrd = new Thread (new InterruptibleThread());
Thrd.start();
```

...

```
// now interrupt it
Thrd.interrupt();
```

## Thread Cancellation (Cont.)

```
public class InterruptibleThread implements Runnable
{
    public void run() {
        while (true) {
            /**
             * do some work for awhile
             */

            if (Thread.currentThread().isInterrupted()) {
                System.out.println("I'm interrupted!");
                break;
            }
        }

        // clean up and terminate
    }
}
```

bring one that available

## Thread Pools

- **Instantiate several threads** in a pool and have them remain idle, waiting for tasks to be assigned.
- Advantages:
  - Typically, handling a request using an existing thread is more efficient than creating a new one.
  - Enables the application(s) to allocate the number of threads based on the size of the pool.

if finish it will go back to the pool.

## Thread Pools

- Java provides 3 thread pool architectures:
  1. **Single thread executor** - pool of size 1.
    - `static ExecutorService newSingleThreadExecutor()`
  2. **Fixed thread executor** - pool of fixed size.
    - `static ExecutorService newFixedThreadPool(int nThreads)`
  3. **Cached thread pool** - pool of unbounded size
    - `static ExecutorService newCachedThreadPool()`

Task.java  
NonTPEExample.java  
TPEExample.java

## CPU Scheduling

## CPU Scheduler

- Chooses from the processes in memory that are ready to run, and assigns the CPU to one of them
- **nonpreemptive** OS can interrupt the process.
- **preemptive**
  - ↓
  - OS can interrupt the running process.

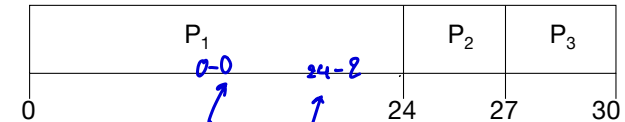
## Scheduling Algorithm Optimization Criteria

- Max CPU utilization
- Max throughput
- Min turnaround time
- Min waiting time
- Min response time

## First-Come, First-Served (FCFS) Scheduling

non-preemptive Algo  $\rightarrow$  First come, first served.

| Process | Arrival Time | Burst Time |
|---------|--------------|------------|
| $P_1$   | 0.0          | 24         |
| $P_2$   | 2.0          | 3          |
| $P_3$   | 4.0          | 3          |



- Waiting time for  $P_1 = 0$ ;  $P_2 = 22$ ;  $P_3 = 23$
- Average waiting time:  $(0 + 22 + 23)/3 = 15$

$$\left( \frac{0 + 22 + 23}{3} \right) \approx 15$$

turn around time = finish time - arrival time

$$TP_1 = 24 - 0 = 24$$

$$TP_2 = 27 - 2 = 25$$

$$TP_3 = 30 - 4 = 26$$

waiting time = turnaround time - burst time

$$WP_1 = 24 - 24 = 0$$

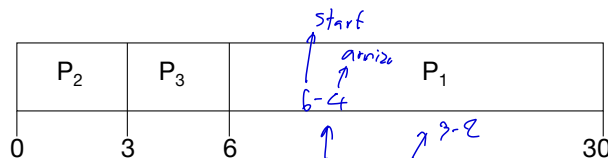
$$WP_2 = 25 - 3 = 22$$

$$WP_3 = 26 - 3 = 23$$

## FCFS Scheduling (Cont)

| Process | Arrival Time | Burst Time |
|---------|--------------|------------|
|---------|--------------|------------|

- 1  $P_2$  0.0 3
- 2  $P_3$  2.0 3
- 3  $P_1$  4.0 24



- Waiting time for  $P_1 = 2$ ;  $P_2 = 0$ ;  $P_3 = 1$
- Average waiting time:  $(2 + 0 + 1)/3 = 1$
- Much better than previous case
- Convoy effect short process behind long process

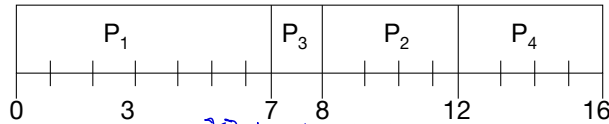
## Shortest-Job-First (SJF) Scheduling

- Assign a value representing the duration of the next CPU burst to each process.
- Utilize these lengths to efficiently schedule the process with the shortest time.
- SJF is optimal – gives minimum average waiting time for a given set of processes
  - It can be challenging to determine the length of the upcoming CPU request

## Example of SJF

| Process        | Arrival Time | Burst Time |
|----------------|--------------|------------|
| P <sub>1</sub> | 0.0          | 7          |
| P <sub>2</sub> | 2.0          | 4          |
| P <sub>3</sub> | 4.0          | 1          |
| P <sub>4</sub> | 5.0          | 4          |

- SJF scheduling chart



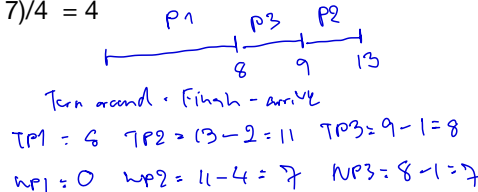
- Average waiting time =  $(0 + 6 + 3 + 7)/4 = 4$

$$TP_1 = 7 - 0 = 7$$

$$TP_2 = 12 - 2 = 10$$

$$TP_3 = 8 - 4 = 4$$

$$TP_4 = 16 - 5 = 11$$



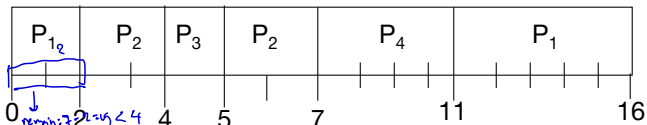
## Shortest Remaining Time First (SRTF)

- Preemptive version of the SJF algorithm
- Assigning the processor to the job that is closest to completion
  - If a newer job in the READY queue has a shorter time to completion, it can preempt the current job
- Cannot be implemented in interactive system
  - Requires advance knowledge of the CPU time required to finish each job
- SRTF has more overhead than SJF
  - The operating system tracks the CPU time for all jobs in the READY queue and handles context switching.

## Example of SRTF

| Process        | Arrival Time | Burst Time |
|----------------|--------------|------------|
| P <sub>1</sub> | 0.0          | 7          |
| P <sub>2</sub> | 2.0          | 4          |
| P <sub>3</sub> | 4.0          | 1          |
| P <sub>4</sub> | 5.0          | 4          |

- SJF (preemptive)



- Average waiting time =  $(9 + 1 + 0 + 2)/4 = 3$

Turn around = Finish - Arrived

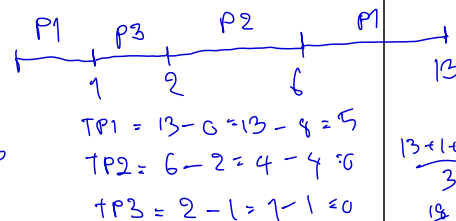
$$TP_1 = 16 - 0 = 16$$

$$TP_2 = 7 - 2 = 5$$

$$TP_3 = 5 - 4 = 1$$

$$TP_4 = 11 - 5 = 6$$

$$\left. \begin{array}{l} \text{Waiting time} = \text{Turn around} - \text{burst time} \\ WP_1 = 16 - 7 = 9 \\ WP_2 = 5 - 4 = 1 \\ WP_3 = 1 - 1 = 0 \\ WP_4 = 6 - 4 = 2 \end{array} \right\} \frac{(9 + 1 + 0 + 2)}{4} = 3$$



## Priority Scheduling

- Each process is assigned a priority value, typically an integer. The priority can be determined based on various criteria, such as the importance of the task, user preference, or resource requirements.
- The CPU is allocated to the process with the highest priority
- Algorithms
  - Preemptive
  - Non-preemptive
- SJF is a priority scheduling where priority is the predicted next CPU burst time
- Problem  $\equiv$  **Starvation** - low priority processes may never execute
- Solution  $\equiv$  **Aging** - as time progresses increase the priority of the process

longer time  $\rightarrow$  will increase the priority of the process.

$$\frac{13 + 1 + 4}{3} = 6 \#$$

10.)

|    |   |   |
|----|---|---|
| P1 | 8 | 7 |
| P2 | 4 |   |
| P3 | 1 |   |

$\overbrace{\quad}^{P1}$   
1

## fine sharing Round Robin (RR)

- Every process is allocated a small amount of CPU time, typically ranging from 10 to 100 milliseconds, known as a time quantum.
- Once this time has passed, the process is interrupted and placed at the end of the ready queue
- If the ready queue contains  $n$  processes and the time quantum is  $q$ , each process is allocated no more than  $q$  time units at a time. No process waits more than  $(n-1)q$  time units.
- Performance
  - $q$  large  $\Rightarrow$  FIFO
  - $q$  small  $\Rightarrow q$  must be large with respect to context switch, otherwise overhead is too high

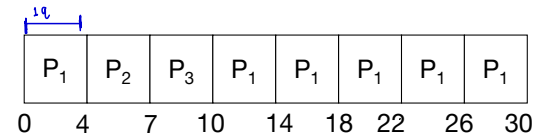
Ex  $P_1, P_2, P_3, P_4$

$n=4$   
 $q = 10 \text{ ms}$      $(n-1)q$   
 $(4-1)(10 \text{ ms}) = 30 \text{ ms}$

## Example of RR with Time Quantum = 4

| Process | Arrival Time | Burst Time |
|---------|--------------|------------|
| $P_1$   | 0.0          | 24         |
| $P_2$   | 2.0          | 3          |
| $P_3$   | 4.0          | 3          |

- The Gantt chart is:



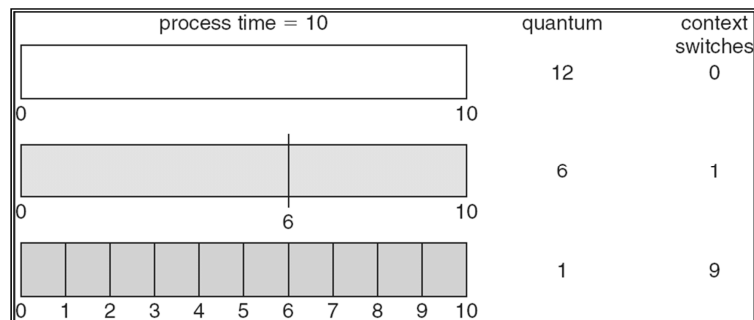
- Typically, higher average turnaround than SJF, but better response

$T_{\text{turn}} = \text{Finish} - \text{arrive}$      $T_{\text{turn}} = \text{wait} + \text{burst}$

$TP_1 = 30 - 0 = 30$   
 $TP_2 = 7 - 2 = 5$   
 $TP_3 = 10 - 4 = 6$

$\text{wait} = T_{\text{turn}} - \text{burst}$   
 $WP_1 = 30 - 24 = 6$   
 $WP_2 = 5 - 3 = 2$   
 $WP_3 = 6 - 3 = 3$

## Time Quantum and Context Switch Time



## Multilevel Queue Scheduling

- Multilevel Queue Scheduling is a scheduling algorithm that partitions the ready queue into several separate queues.
- Each queue can have its own scheduling algorithm. For Examples
  - foreground - RR  $\rightarrow$  interact with user
  - background - FCFS  $\rightarrow$  batch
- This method is used when processes can be easily categorized into different classes with different scheduling needs.
- No Movement Between Queues: Processes do not move between queues. Once assigned to a queue, a process stays there.

## Multilevel Queue Scheduling (Cont.)

- Scheduling must be done between the queues
  - Fixed priority scheduling; (i.e., serve all from foreground then from background). Possibility of starvation.
  - Time slice – each queue gets a certain amount of CPU time which it can schedule amongst its processes; i.e., 80% to foreground in RR 20% to background in FCFS

avoid starvation

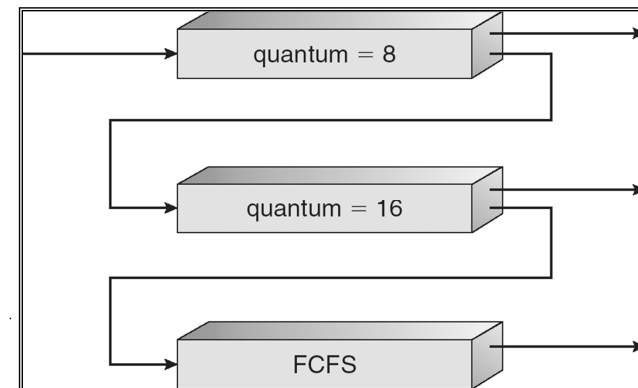
## Multilevel Feedback Queue Scheduling

- A process can move between the various queues; aging can be implemented this way
- Multilevel-feedback-queue scheduler defined by the following parameters:
  - number of queues
  - scheduling algorithms for each queue
  - method used to determine when to upgrade a process
  - method used to determine when to demote a process
  - method used to determine which queue a process will enter when that process needs service

## Example of Multilevel Feedback Queue

- Three queues:
  - $Q_0$  – RR with time quantum 8 milliseconds
  - $Q_1$  – RR time quantum 16 milliseconds
  - $Q_2$  – FCFS
- Scheduling
  - A new job enters queue  $Q_0$  which is served FCFS. When it gains CPU, job receives 8 milliseconds. If it does not finish in 8 milliseconds, job is moved to queue  $Q_1$ .
  - At  $Q_1$  job is again served FCFS and receives 16 additional milliseconds. If it still does not complete, it is preempted and moved to queue  $Q_2$ .

## Multilevel Feedback Queues



## Java Thread Scheduling

- Java specification does not specify the standard scheduling.
- It depends on the implementation of each JVM.
- Do not rely on the scheduling algorithm for the correctness of your program.

## Java Threads Scheduling (Cont.)

- Java typically employs a preemptive and priority-based scheduling method.
- Every Java thread possesses a priority. The thread with the greatest priority is selected as the first choice.
- When multiple threads with the same priority await the CPU, the scheduler selects one to execute randomly.

## Java Threads Scheduling (Cont.)

- The chosen thread runs until one of the following conditions is true:
  - A higher priority thread becomes runnable.
  - It yields, or its run method exits.
  - On systems that support time-slicing, its time slot has expired.

## Java Threads Scheduling (Cont.)

- Priority is in the range of 1-10.
- The default priority is 5.
- Method `setPriority()` is used to set a thread priority.
  - `myThread.setPriority(6);`
- A thread can yield control of the CPU using the `yield()` method.
  - Java can then choose another thread with the same priority to run.

